AWS X-Rays

Explore how AWS X-Ray provides distributed tracing for cloud-native applications. Learn to track requests across multiple services to identify performance bottlenecks and errors, improving application observability and troubleshooting efficiency.

Modern cloud-native architectures often consist of multiple microservices working together to fulfill a single user request. Pinpointing the root cause can feel like untangling spaghetti when something goes wrong. Distributed tracing helps unravel this complexity by showing how requests flow through our services.

Distributed tracing helps unravel this complexity by showing a request's complete, end-to-end journey as it flows through each service in an application.

This differs from traditional logging, which provides individual snapshots of events at specific moments. Logs can be unstructured, stored in various locations, and often lack connections between services. Tracing, on the other hand, gives us a contextual view of the entire request flow. It links together how multiple services collaborate to process a single request. While we still need logs for detailed debugging and auditing, tracing offers an overall perspective that logging alone cannot provide.

In AWS, X-Ray provides a powerful way to implement distributed tracing, offering deep insights into application behavior, performance bottlenecks, and service dependencies.

Ask

However, before exploring AWS X-Ray, let's first understand what distributed systems are.

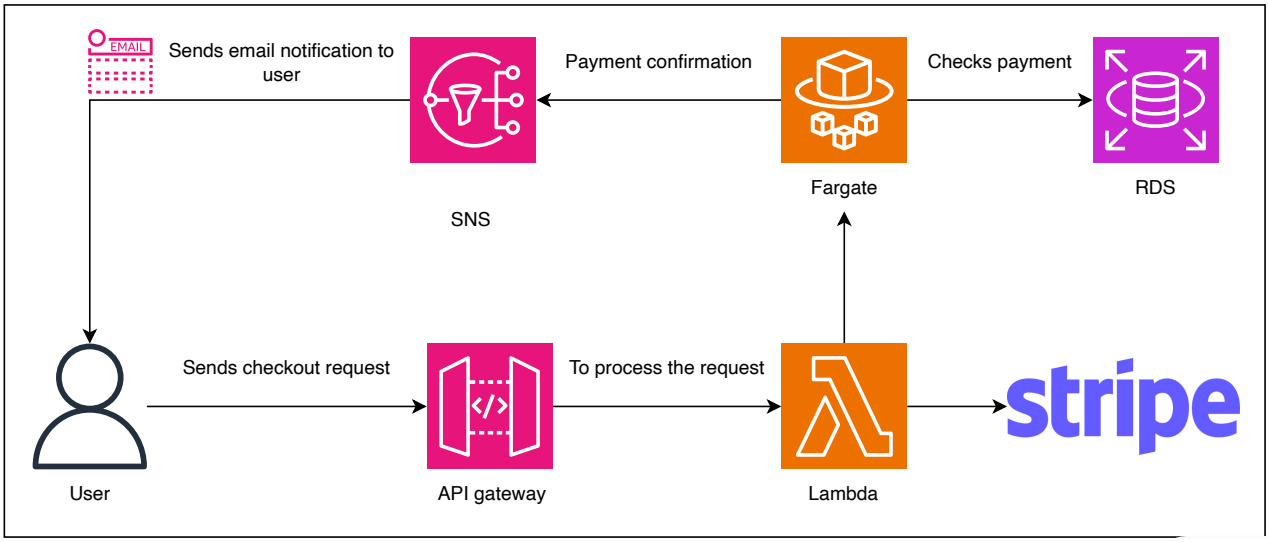
Distributed systems: Checkout flow for an e-commerce application

Let's explore a common scenario in distributed systems using an e-commerce checkout flow to understand the challenges and how distributed tracing helps.

Imagine being part of a development team managing an e-commerce application on AWS. The checkout process seems straightforward to the user, but behind the scenes, it involves a complex series of interactions across multiple services.

Here's the typical flow:

1. An API Gateway endpoint receives the checkout request.
2. This triggers a Lambda function in the `order-service`, which processes payments using Stripe.
3. Once payment is successful, the `inventory-service` (hosted on Fargate and backed by Amazon RDS) confirms stock and locks the items.
4. Finally, a `notification-service` sends a confirmation email via SNS and Lambda.



Infrastructure for the checkout flow

Ask

On the surface, everything appears fine. However, users start reporting that their orders aren't going through, with no error messages on the frontend. The logs for the `order-service` indicate that the request was processed, and Stripe confirms successful transactions. Yet, some orders simply disappear; they never make it to the database or trigger email confirmations.

We check CloudWatch Logs across services, but nothing obvious stands out. Each service is logging its part of the process, but we lack the complete picture—the full flow of the request from one service to the next. This leaves us wondering: Where exactly is the request getting lost? This is the kind of ambiguity and difficulty in pinpointing the root cause that distributed tracing is designed to resolve.

Distributed tracing

Distributed tracing is a method of observing and tracking the flow of a request as it travels through the various components of a distributed system, across APIs, containers, databases, and queues. It allows us to understand how each service participates in handling a user's request and how much time each component takes.

In a distributed trace, every operation that handles part of a request is captured as a **span** or **segment**. Each span records essential metadata such as:

- **Start time and duration:** When the operation began and how long it took.
- **Component name:** The service or component involved (e.g., `OrderService-Lambda`).
- **Operation name:** A description of the action performed (e.g., `POST /checkout`).
- **HTTP status code or error:** The result of the operation.
- **Tags or annotations:** Additional relevant information (e.g., `order ID`).

Ask

Spans are grouped together under a single trace ID, forming a trace that shows a request's end-to-end journey. Considering our e-commerce checkout flow. A trace for it might look like this:

index.yaml YAML

```
Trace: POST /checkout (Total Duration: ~950ms)
└── Span A: OrderService-Lambda (duration: 900ms)
    ├── Span B: Stripe.processPayment (duration: 600ms)
    ├── Span C: InventoryService.confirmStock (Fargate) (duration: 25
    │   └── Span D: RDS.updateInventory (duration: 200ms)
    └── Span E: SNS.publishConfirmation (duration: 50ms)
```

Sample trace for checkout flow

This trace clearly shows the sequence of operations and their individual durations, making it much easier to pinpoint where delays or issues occur within the complex system. The diagram below illustrates how a single trace consists of multiple spans.

Ask

Spans and traces

A trace shows the complete journey of a user request across multiple services, with each part of that journey recorded as a span. These spans are organized like a tree, where one parent span can start several child spans.

Within a parent span, child spans can run either sequentially (one after another) or concurrently (at the same time). Regardless of how they execute, all spans within a single trace are connected to the same overall request. All of these spans are recorded under the same trace ID.

Types of distributed tracing

There are typically three types of tracing we encounter in practice.

- **Program tracing** captures the fine-grained execution flow of an application, line by line or function by function. It is often used during debugging or runtime analysis. While it provides insights into control flow, recursion, how algorithms execute, it doesn't scale well in distributed environments.

Ask

- **Code tracing** focuses on tracing important operations within a single service or module. This includes things like handling HTTP requests, interacting with databases, or making calls to third-party APIs. This type of tracing helps us understand the internal behavior of an isolated service and is often used to pinpoint bottlenecks or failures within that specific service.
- **End-to-end tracing** tracks a request as it travels across multiple services in a distributed system. It covers the entire journey, from the initial entry point (like a user click or an API call) through all the microservices, databases, and event queues involved. This is the primary focus of distributed tracing tools such as AWS X-Ray, and it offers the *clearest picture* of how an entire system collaborates to fulfill a request.

Components of distributed tracing

To make distributed tracing possible, a few standard components work together:

- **Tracer SDK or agent:** This component is integrated directly into an application's code. Its job is to capture spans and relevant metadata as the application processes requests.
- **Trace ID propagation:** This is how a unique identifier for a trace is passed from one service to the next as a request moves through the system. This is typically done using HTTP headers to ensure that all parts of a single request are linked together under the same trace.
- **Segments or spans:** In AWS X-Ray specifically, segments represent the work performed by an individual service. Within a segment, subsegments can be used to capture more detailed, finer-grained operations, such as database calls or external HTTP requests made by that service.
- **Trace store and visualizer:** This is a central service, like AWS X-Ray, that stores all the collected trace data. It then provides a user interface to visualize this data, often in the form of timelines showing the duration of each

Ask

operation or service maps illustrating the relationships and interactions between different services in our application.

What is AWS X-Ray?

AWS X-Ray is a distributed tracing service that helps developers and system administrators analyze and debug applications, particularly those built using a microservices architecture. X-Ray enables us to visualize and track requests as they travel through various components of an application, helping to identify and diagnose performance issues, errors, and bottlenecks.

How X-Ray works

To monitor an application using AWS X-Ray, we must first instrument it. In AWS X-Ray, **instrumenting** refers to the process of adding tracing code to the application so that it can send data to X-Ray.

AWS X-Ray begins the process of tracing with trace collection. Data is collected from each service in an instrumented application. The trace data includes metadata about incoming and outgoing requests. To track requests as they move between different services, X-Ray uses HTTP headers. This method allows X-Ray to build a detailed view of a request's journey across the entire application.

Once collected, the trace data is organized into traces. Each trace represe single request as it flows through various application components. These traces capture important details such as latency, HTTP response status, and error

Ask

counts. This structured data provides a clear picture of how each application part performs.

Working of X-Ray

Trace details provide an in-depth view of a specific request's journey through our services. When we expand a trace, we can see a map of the service nodes, illustrating how they interacted to fulfill that particular request. It also includes a segment timeline, which lists all the segments and sub-segments in chronological order. Each segment and sub-segment displays its duration, status, and status code. If applicable, it also shows the namespace or environment where the segment or sub-segment executed. This level of detail helps us pinpoint performance bottlenecks or identify where errors occurred within the reque

Ask

How to enable X-Ray in an application

Enabling X-Ray in an application involves instrumenting the AWS services we use and, for some, adding tracing code directly to our application. For many AWS-managed compute services like Lambda, API Gateway, and Step Functions, enabling tracing is straightforward, often just a matter of flipping a switch.

For instance, when setting up a Lambda function, we can enable **active tracing** directly in the AWS console or by using Infrastructure as Code (IaC) tools like CloudFormation or Terraform. This setting allows Lambda to automatically create segments for its execution and pass trace headers to downstream services.

For services running on EC2, ECS, or custom code outside managed environments, we'll need to install and run the X-Ray daemon. Additionally, we must manually instrument our code using one of the AWS X-Ray SDKs.

Let's walk through a basic example of how to integrate X-Ray into a Python service using the AWS X-Ray SDK for Python, `aws-xray-sdk`.

main.py Python

```python
from aws_xray_sdk.core import xray_recorder
from aws_xray_sdk.core import patch_all

# Automatically patch supported libraries (e.g., requests, boto3, etc
patch_all()

def handle_checkout(event):
    xray_recorder.begin_segment('CheckoutService')

    try:
        process_payment(event['order_id'])
    finally:
        xray_recorder.end_segment()
```

Ask

Python code to use aws-xray-sdk

The example demonstrates how to instrument a Python service with AWS X-Ray.

- We use `patch_all()` on **line 5** to automatically trace calls to supported libraries.
- The `handle_checkout` function begins an X-Ray segment for the entire checkout process, while `process_payment` on **line 11** creates a subsegment for the `StripeAPI` call.
- We also see how to add custom annotations like `order_id` to subsegments for better filtering and analysis in X-Ray.

Visualizing trace data with AWS X-Ray

After AWS X-Ray collects and structures trace data, it offers visual tools to help us understand key insights. These visualizations are often the starting point for debugging complex issues in production.

Service map

The **service map** is one of X-Ray's most powerful features. It gives us a high-level, real-time view of how our application's components are connected and how they are performing. Every node in the map represents a service, such as a Lambda function, an ECS task, or a DynamoDB table, and the edges between nodes represent interactions between services.

What makes the service map so valuable is that it overlays real-time performance indicators. Nodes are color-coded based on latency and error rates. For example, a red outline might signal high error rates in a service, while a yellow border could indicate growing latency. This visual snapshot helps us instantly identify performance hotspots and service failures, even when they are buried deep in the call chain.

Ask

Sample service map

Trace view

While the service map provides a high-level overview, the **trace view** allows us to zoom in on individual requests. A trace view represents a single request and displays all the segments and subsegments involved in processing that request. This view is rendered as a waterfall timeline, where each bar corresponds to the time taken by a service or internal operation.

For example, in our checkout flow use case, a single trace might show that the `order-service` Lambda ran for 120 ms, then waited 300 ms on the `inventory-service`, which ultimately failed with a time-out. These details are often the only way we can truly understand what's going wrong across asynchronous or event-driven systems.

Ask

These trace views are used for diagnostics and as a learning tool. They show how services are chained together and how even small latencies add up in multi-hop

architectures.

Understanding the X-Ray daemon

The **X-Ray daemon** is a lightweight process that acts as a middle layer between our application and the AWS X-Ray service.

When our application creates trace segments, either through manual SDK instrumentation or automatically via AWS integrations, it sends them to the daemon over UDP (User Datagram Protocol). The daemon then batches these segments and sends them to the X-Ray service over HTTPS. This batching approach is efficient and prevents overwhelming the network or the tracing backend.

We can run the daemon as a background process on EC2, ECS, or even inside containers. For Lambda, X-Ray integration is automatic, with the daemon embedded and managed by AWS.

The code snippet given below shows the CloudFormation template to create a Lambda function with tracing enabled:

index.yaml YAML

```yaml
Resources:
  MyLambdaFunction:
    Type: AWS::Lambda::Function
    Properties:
      FunctionName: myLambdaFunction
      Handler: index.handler
      Role: arn:aws:iam::123456789012:role/lambda_exec_role
      Runtime: python3.9
      Code:
        S3Bucket: my-bucket
        S3Key: lambda_function_payload.zip
      TracingConfig:
        Mode: Active
```

Ask

CloudFormation template to create a Lambda function with tracing enabled

The daemon is important because it decouples trace generation from trace delivery. This means our application can quickly send trace data without having to wait for it to be fully transmitted or stored by the X-Ray service. This separation improves performance and reliability, ensuring that tracing operations don't negatively impact our production traffic.

Ask